

detonationFoam for OpenFOAM Foundation 14

Theory, Usage, OF8 Migration, AMR, Parallelism, and Release Qualification

Version 1.0.0 - September 2026

Release scope. This manual documents the modular OpenFOAM Foundation 14 port of `detonationFoam_V2.0_OF8`. The solver is delivered as the `foamRun` module `detonationFluid`. The reusable two-dimensional AMR component is `planarRefiner`, an `fvMeshTopoChanger` in `libplanarFvMeshTopoChangers.so`. It is **not** an `fvModel` and may be built and used without `detonationFluid`.

Table of Contents

1. Introduction.....	4
2. Package architecture.....	4
2.1 Solver module.....	4
2.2 Legacy transport compatibility library.....	4
2.3 Reusable 2-D/wedge AMR library.....	5
3. Governing theory.....	5
3.1 Continuity:.....	5
3.2 Momentum:.....	5
3.3 Species:.....	5
3.4 Total-energy form: with chemical heat release.....	6
3.5 Thermodynamics:.....	6
3.6 Acoustic time-step control:.....	6
4. Numerical flux families.....	6
4.1 Runtime selection: fluxScheme.....	6
4.2 Reconstruction.....	7
5. Chemistry and transport.....	7
5.1 OpenFOAM 14 finite-rate chemistry.....	7
5.2 Euler.....	7
5.3 NS_Sutherland.....	7
5.4 Exact legacy NS_mixtureAverage behavior.....	7
5.5 Soret/thermal diffusion.....	8
6. Dynamic mesh, AMR, and load balancing.....	8
6.1 Native 3-D AMR.....	8
6.2 Reusable 2-D/axisymmetric AMR: planarRefiner.....	9
6.3 Native load balancing.....	9

Table of Contents (continued)

6.4 Restart.....	10
7. Building and installing.....	10
7.1 Prerequisite.....	10
7.2 Full detonation solver build.....	10
7.3 AMR-only build.....	10
8. Running a case.....	10
8.1 Minimum controlDict.....	10
8.2 Solver-type selection.....	11
8.3 Flux selection.....	11
8.4 Chemistry.....	11
8.5 Serial run.....	11
8.6 Parallel run.....	11
9. Migrating an OpenFOAM 8 detonationFoam case to OpenFOAM 14.....	11
9.1 Migration workflow.....	11
9.2 OF8 to OF14 capability disposition.....	12
9.3 Important migration detail for NS_mixtureAverage.....	13
10. Qualification summary.....	13
11. Known limitations and post-release work.....	14
12. Troubleshooting.....	15
12.1 foamRun cannot select detonationFluid.....	15
12.2 legacyMixtureAverageFourier is not found.....	15
12.3 AMR library is not found.....	15
12.4 Load balancer says Selecting distributor simple.....	15
12.5 Localized 2-D AMR creates a few non-hex transition cells.....	15
13. File map.....	15

1. Introduction

The OpenFOAM 14 port preserves the density-based reacting detonation formulation and the legacy detonation flux-family choices while adopting the modular OpenFOAM 14 `foamRun` architecture. The primary solver library is:

```
libdetonationFluidSolver.so
```

and cases select it with:

```
solver detonationFluid;
```

in `system/controlDict`, then execute with:

```
foamRun
```

The port deliberately uses OpenFOAM 14 native capability where it is equivalent and maintains targeted compatibility code only where the OpenFOAM 8 behavior was not reproduced by a stock OpenFOAM 14 model. The largest compatibility layer is the exact legacy `NS_mixtureAverage` transport/property path, implemented through the standard OpenFOAM 14 multicomponent thermophysical-transport interfaces rather than by restoring a separate legacy solver branch.

The release also contains a reusable dynamic 2-D/axisymmetric refiner:

```
libplanarFvMeshTopoChangers.so
```

with runtime type:

```
planarRefiner
```

This AMR library is intentionally independent of `detonationFoam`.

2. Package architecture

2.1 Solver module

`applications/modules/detonationFluid/` contains the `foamRun` solver module. Its architecture follows the OpenFOAM 14 density-based shock-fluid pattern while adding reacting multicomponent chemistry, the restored detonation flux families, and optional viscous/multicomponent transport.

The main field set is:

- pressure p ;
- density ρ ;
- velocity U ;
- mass flux ϕ ;
- specific internal energy e ;
- species mass fractions Y_i ;
- chemical heat-release rate $\dot{Q}$.

`fvModels` and `fvConstraints` remain available through normal OpenFOAM 14 interfaces in the density, momentum, species, and energy equations.

2.2 Legacy transport compatibility library

`src/detonationLegacyThermophysicalTransportModels/legacyMixtureAverageFourier/` builds:

```
libdetonationLegacyThermophysicalTransportModels.so
```

This library provides:

- `legacyMixtureAverageFourier`;
- `legacyBinaryDiffusionCoefficient`;
- the runtime registration needed for the converted `logPolynomial` species transport data with `coefficientWilkeMulticomponentMixture`.

It is used only when exact legacy transport/property behavior is desired.

2.3 Reusable 2-D/wedge AMR library

`src/planarFvMeshTopoChangers/` builds:

`libplanarFvMeshTopoChangers.so`

The runtime type is `planarRefiner`. The library owns topology refinement for slab/empty and wedge/axisymmetric meshes and contains no dependency on `detonationFluid`.

Terminology note: the 2-D AMR component is sometimes informally called the “2-D AMR fvModel”. In OpenFOAM 14 it is correctly implemented as an **`fvmeshTopoChanger`**, because topology change is a mesh operation. `fvmesh` remains the equation/source-term abstraction.

3. Governing theory

3.1 Continuity: $\partial_t \rho + \nabla \cdot (\rho U) = S_\rho$

The density equation advanced by `correctDensity.C` is

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho U) = S_\rho,$$

where S_ρ represents any source supplied through `fvmeshModels().source(rho)`. The face mass flux is constructed by the selected shock-capturing flux backend rather than by a pressure-based flux correction.

3.2 Momentum: $\partial_t (\rho U) + \nabla \cdot F_{\rho U} = \nabla \cdot \tau + S_U$

The conservative momentum equation is represented as

$$\frac{\partial (\rho U)}{\partial t} + \nabla \cdot F_{\rho U} = \nabla \cdot \tau + S_U.$$

For `Euler`, the viscous stress term is disabled. For the viscous path, the stress operator comes from the OpenFOAM 14 compressible momentum-transport model through `momentumTransport_->divDevTau(U)`.

The numerical momentum flux contains both the transported momentum and the pressure contribution. HLLCP additionally supplies explicit conservative pressure-difference corrections.

3.3 Species: $\partial_t (\rho Y_i) + \nabla \cdot (\phi Y_i) = \dot{\omega}_i - \nabla \cdot j_i + S_i$

For each solved species,

$$\frac{\partial (\rho Y_i)}{\partial t} + \nabla \cdot (\rho U Y_i) = \dot{\omega}_i - \nabla \cdot j_i + S_i,$$

where $\dot{\omega}_i$ is the finite-rate chemistry source, j_i is the diffusive mass flux for viscous modes, and S_i is an optional `fvmesh` source.

After the species equations are solved, OpenFOAM’s multicomponent thermo normalizes the mass fractions:

$$\sum_i Y_i = 1.$$

The qualification suite explicitly checks this closure after transport, AMR mapping, MPI redistribution, and restart.

3.4 Total-energy form: $e + K$ with chemical heat release

The solver advances sensible internal energy e together with the kinetic-energy contribution $K = |U|^2/2$. In continuous notation the implemented balance corresponds to

$$\frac{\partial(\rho e)}{\partial t} + \nabla \cdot F_E + \frac{\partial(\rho K)}{\partial t} = \dot{Q} + S_e + \nabla \cdot q_{diff} + \nabla \cdot (\tau \cdot U),$$

with the diffusive terms omitted for Euler. The convective energy flux contains reconstructed internal energy, kinetic energy, and the pressure-work flux supplied by the selected Riemann/central-upwind backend. Mesh-motion pressure work is included for moving meshes.

The chemistry model supplies the chemical heat-release rate

$$\dot{Q} = \dot{Q}_{chem},$$

which is obtained from the OpenFOAM chemistry interface `reaction_->Qdot()`.

3.5 Thermodynamics: $p = \rho/\psi$

After the conservative update, pressure is recovered from the OpenFOAM thermodynamic compressibility ψ :

$$p = \frac{\rho}{\psi}.$$

The converted reference mechanism uses a perfect-gas equation of state with JANAF thermochemistry and sensible internal energy. The exact legacy transport case uses:

```
mixture      coefficientWilkeMulticomponentMixture;
transport    logPolynomial;
thermo       janaf;
energy       sensibleInternalEnergy;
equationOfState perfectGas;
```

3.6 Acoustic time-step control: Co_a

The density-based time-step limit is based on an acoustic face signal speed. In compact form,

$$Co_a = \frac{1}{2} \Delta t \max_c \left[\frac{\sum_{f \in c} a_{max,f} |S_f|}{V_c} \right].$$

The local-time-stepping form uses the same signal-speed measure to set the reciprocal time scale. Standard cases use `adjustTimeStep yes` and `maxCo` in `controlDict/PIMPLE` settings.

4. Numerical flux families

4.1 Runtime selection: `fluxScheme`

The flux backend is selected in `system/fvSchemes`:

```
fluxScheme HLLCP;
```

The release supports seven backends:

Flux	Release status	Role
Kurganov	qualified	central-upwind baseline
Tadmor	qualified	central scheme
HLL	restored and qualified	two-wave HLL Riemann flux
HLLC	restored and qualified	HLL with contact-wave restoration
HLLCP	restored and qualified	pressure-corrected HLLC-family detonation flux
AUSM+	restored and qualified	AUSM-family convective/pressure splitting
AUSM+up	restored and qualified	AUSM+ with pressure/velocity dissipation controls

All backends feed a unified conservative OpenFOAM 14 face-flux path. The HLLC/HLLCP implementations retain contact/star-state logic without creating a separate solver branch.

4.2 Reconstruction

The reference cases use OpenFOAM 14 `shockFluid`-style reconstruction keys, for example:

```
reconstruct(rho)    Minmod;
reconstruct(U)      MinmodV;
reconstruct(T)      Minmod;
```

The species and thermodynamic fields use the common face directions generated by the selected flux backend.

5. Chemistry and transport

5.1 OpenFOAM 14 finite-rate chemistry

The legacy DLBFoam chemistry/load-balancing implementation is not carried forward as a separate chemistry engine. OpenFOAM 14 standard chemistry is used instead:

```
type      standard;
chemistry on;
cpuLoad   true;
```

`cpuLoad true` registers chemistry cost information that can be consumed by the native OpenFOAM 14 `loadBalancer` in parallel cases.

5.2 Euler

`solverType Euler`; disables viscous momentum, species diffusion, and conductive/diffusive energy transport while retaining finite-rate chemistry and the selected shock-capturing flux.

5.3 NS_Sutherland

`solverType NS_Sutherland`; activates the viscous transport path. For a conventional Sutherland case, native OpenFOAM 14 momentum/thermophysical transport is used. The fixed-time OF8-to-OF14 comparison for this path passed with identical shock location and small profile differences.

5.4 Exact legacy NS_mixtureAverage behavior

The OpenFOAM 8 `NS_mixtureAverage` equation branch is **not** restored as a separate solver type. Instead, its missing physics is implemented through normal OpenFOAM 14 runtime interfaces.

For species i , the legacy mixture-averaged diffusion coefficient is

$$D_i = \frac{1 - Y_i}{\sum_{j \neq i} \frac{X_j}{D_{ij}}}.$$

The release evaluates the raw positive diffusion vector

$$F_i = \rho D_i \nabla Y_i + Y_i \rho D_i \frac{\nabla W_{mix}}{W_{mix}},$$

and exposes the corrected physical species flux

$$j_i = -F_i + Y_i \sum_k F_k,$$

which enforces

$$\sum_i j_i = 0$$

up to numerical roundoff.

The exact OF8 binary-diffusion correlation is

$$D_{ij}(p, T) = 10^{-4} \frac{\exp \left[D_1 + \ln(T) \left(D_2 + \ln(T) \left(D_3 + \ln(T) D_4 \right) \right) \right]}{p/101325},$$

with p in Pa, T in K, and D_{ij} in $m^2 s^{-1}$.

Species viscosity and conductivity use the converted `muLogCoeffs<8>` and `kappaLogCoeffs<8>` through native `OpenFOAM 14 logPolynomialTransport`. Mixture viscosity uses native `coefficientWilkeMulticomponentMixture`, which was verified algebraically identical to the legacy Wilke rule.

Legacy mixture conductivity is retained as the arithmetic/harmonic average

$$\lambda_A = \sum_i X_i \lambda_i, \lambda_B = \sum_i \frac{X_i}{\lambda_i},$$

$$\lambda_{mix} = \frac{1}{2} \left(\lambda_A + \frac{1}{\lambda_B} \right).$$

A case activates this path with `legacyMixtureAverageFourier` in `constant/thermophysicalTransport` and loads `libdetonationLegacyThermophysicalTransportModels.so` from `controlDict`.

5.5 Soret/thermal diffusion

The release keeps legacy Soret transport **off**. The published OF8 source contains an inconsistent H/H2 thermal-diffusion assignment, so enabling a literal legacy mode would preserve an apparent implementation ambiguity. A corrected Soret model or explicitly named forensic-compatibility mode is deferred.

6. Dynamic mesh, AMR, and load balancing

6.1 Native 3-D AMR

For genuine 3-D cases, OpenFOAM 14's native `fvmMeshTopoChangers::refiner` is the preferred path. The migration qualified serial refinement, fixed-mesh versus localized-AMR behavior, MPI redistribution, and restart.

6.2 Reusable 2-D/axisymmetric AMR: `planarRefiner`

The release adds the solver-independent `planarRefiner` for true 2-D meshes. It supports:

- slab/empty geometry;
- wedge/axisymmetric geometry;
- localized scalar-field band selection;
- MPI distribution callbacks;
- restart state after refinement;
- operation with stock `incompressibleFluid`, proving portability beyond `detonationFoam`.

For one planar refinement pass, each selected parent is split in the two active geometric directions into four children, giving the exact global growth bound

$$N_{new} = N_{old} + 3 N_{selected}.$$

A typical dictionary is:

```
FoamFile
{
    format    ascii;
    class     dictionary;
    object    dynamicMeshDict;
}

topoChanger
{
    type                planarRefiner;
    libs                ("libplanarFvMeshTopoChangers.so");
    geometry            slab;        // auto | slab | wedge
    refineInterval      1;
    field               T;
    lowerRefineLevel     800;
    upperRefineLevel    4900;
    maxCells            200000;
    maxRefinementIterations 1;
}
```

`geometry auto` detects supported geometry from the boundary patch types. The current release is refinement-only; automatic unrefinement/coarsening is not included.

See the separate `planarRefiner_OF14_Manual` for the complete AMR theory and usage guide.

6.3 Native load balancing

Parallel dynamic load balancing uses OpenFOAM 14's native distributor:

```
distributor
{
    type                loadBalancer;
    libs                ("libfvMeshDistributors.so");
    redistributionInterval 5;
    maxImbalance        0;
    multiConstraint      false;
}
```

The redistribution method is taken from `system/decomposeParDict`. If Scotch redistribution is intended, the active dictionary during `foamRun` must contain:

```
method scotch;
```

The G4 qualification deliberately used a deterministic `simple split` for initial `decomposePar`, then switched the active dictionary to `scotch` before constructing the runtime load balancer.

6.4 Restart

OpenFOAM owns the distributed mesh and registered field state. `planarRefiner` stores only its global refinement-iteration counter in:

```
<time>/polyMesh/planarRefinerState
```

This prevents an already-refined restart from applying the same refinement iteration again. Refined/distributed restart was qualified against a continuous MPI trajectory.

7. Building and installing

7.1 Prerequisite

Source OpenFOAM Foundation 14 so that:

```
echo "$WM_PROJECT_VERSION"
```

returns 14.

7.2 Full detonation solver build

From the release root:

```
./Allwmake
```

This builds:

```
$FOAM_USER_LIBBIN/libdetonationLegacyThermophysicalTransportModels.so  
$FOAM_USER_LIBBIN/libdetonationFluidSolver.so
```

The optional 2-D/wedge AMR library is deliberately not required by this build.

7.3 AMR-only build

Build the reusable AMR component without `detonationFoam`:

```
./AllwmakeAMR
```

or:

```
cd src/planarFvMeshTopoChangers  
./Allwmake
```

This builds only:

```
$FOAM_USER_LIBBIN/libplanarFvMeshTopoChangers.so
```

8. Running a case

8.1 Minimum `controlDict`

```
solver          detonationFluid;
```

```
libs
```

```
(  
    "libdetonationLegacyThermophysicalTransportModels.so" // only when needed  
);
```

For Sutherland/native transport, the compatibility library can be omitted if no compatibility Function2 or runtime registration is required by the case.

8.2 Solver-type selection

In `constant/solverTypeProperties`:

```
solverType      NS_Sutherland;
SW_position_limit 0.099999;
shockDetectionPressure 101425;
stopAtShockPosition false;
```

Supported `solverType` selectors are Euler and NS_Sutherland. Exact legacy NS_mixtureAverage transport is selected through `physicalProperties` and `thermophysicalTransport`; it is not a third OF14 solver branch.

8.3 Flux selection

In `system/fvSchemes`:

```
fluxScheme HLLCP;
```

Choose one of Kurganov, Tadmor, HLL, HLLC, HLLCP, AUSM+, or AUSM+up.

8.4 Chemistry

A typical OpenFOAM 14 chemistry dictionary is:

```
type      standard;
chemistry on;
cpuLoad    true;

initialChemicalTimeStep 1;

ode
{
    solver  seulex;
    absTol  1e-06;
    relTol  1e-03;
}
```

Mechanism data can be included from converted `foam/reactions.foam` and `foam/species.foam/thermo.foam` files.

8.5 Serial run

```
blockMesh
foamRun
```

The included fast tutorial can be run with:

```
cd tutorials/1D_NH3_O2_cracking_0.3_detonation_OF14_fast
./Allrun.serial
```

8.6 Parallel run

Prepare `system/decomposeParDict`, then:

```
decomposePar -force
mpirun -np 4 foamRun -parallel
```

For native runtime load balancing, add the `distributor` block to `constant/dynamicMeshDict` and ensure the active `decomposeParDict` specifies the desired runtime method, such as `scotch`.

9. Migrating an OpenFOAM 8 detonationFoam case to OpenFOAM 14

9.1 Migration workflow

A practical migration sequence is:

1. Copy the mesh and initial fields, preserving the physical initialization rather than re-creating the shock unless a new setup is intended.
2. Replace the standalone solver executable entry with solver detonationFluid; and run with foamRun.
3. Convert thermophysicalProperties to OpenFOAM 14 physicalProperties plus the standard multicomponent thermo data files.
4. Convert chemistry setup to OpenFOAM 14 reactionProperties and chemistryProperties using type standard;.
5. Move flux-family selection to system/fvSchemes with fluxScheme.
6. Retain Euler or NS_Sutherland in constant/solverTypeProperties. For legacy NS_mixtureAverage, configure the compatibility transport model rather than selecting a legacy solver branch.
7. Translate legacy binary-diffusion coefficients to the legacyBinaryDiffusionCoefficient Function2 entries when exact equivalence is required. The release includes tools/ConvertLegacyBinaryDiff.py.
8. Replace legacy/DLBFoam load balancing with the native OpenFOAM 14 loadBalancer and cpuLoad true chemistry accounting.
9. For 3-D AMR, use native OpenFOAM 14 refiner; for true 2-D or wedge AMR, optionally build and select planarRefiner.
10. Run a short fixed-time comparison before extending the simulation duration.

9.2 OF8 to OF14 capability disposition

OpenFOAM 8 detonationFoam capability	OpenFOAM 14 release treatment	Classification	Notes
Standalone detonationFoam executable	detonationFluid runtime solver module executed by foamRun	Translated	Modular OF14 solver architecture
Density-based shock solver structure	OF14 basicFluidSolver/shock-fluid style modular infrastructure	OF14 native substitution	Same density-based design intent; OF14 lifecycle and mesh hooks
Kurganov flux	Retained in unified OF14 face-flux path	OF14/native-style retained	Regression baseline
Tadmor flux	Retained in unified OF14 face-flux path	OF14/native-style retained	Qualified
HLL	Restored in detonationFluid	Translated from OF8	Conservative OF14 implementation
HLLC	Restored in detonationFluid	Translated from OF8	Contact-wave/star-state logic retained
HLLCP	Restored in detonationFluid	Translated from OF8	Pressure-corrected detonation flux retained
AUSM+	Restored in detonationFluid	Translated from OF8	Qualified
AUSM+up	Restored in detonationFluid	Translated from OF8	Qualified
Euler solver type	solverType Euler	Translated/retained	Inviscid transport path
NS_Sutherland	OF14 viscous momentum and thermophysical transport	OF14 native substitution	OF8/OF14 fixed-time profile comparison qualified
Separate solverTypeNS_mixtureAverage equation files	Compatibility model behind OF14 divj()/divq() interfaces	Translated into OF14 runtime model	No duplicate solver branch
Legacy mixture-average D_i law	legacyMixtureAverageFouri	Translated exactly	Uses $(1 - Y_i)$ numerator
Legacy $D_{ij}(p, T)$ Diff1..Diff4 law	legacyBinaryDiffusionCoefficient Function2	Translated exactly	Tool supplied for conversion
Legacy log-polynomial species μ_i, λ_i	Native OF14 logPolynomialTransport <8>	OF14 native substitution	Converted coefficients evaluated directly
Legacy Wilke mixture viscosity	Native coefficientWilkeMulticomponentMixture	OF14 native substitution	Algebraic match verified to machine precision
Legacy arithmetic/harmonic mixture conductivity	legacyKappa() in detonationFoam	Translated exactly	Native Wilke conductivity is not equivalent

OpenFOAM 8 detonationFoam capability	OpenFOAM 14 release treatment	Classification	Notes
	compatibility transport		
Species sensible-enthalpy diffusion	OF14 multicomponent divq() interface	OF14 native framework + translated closure	Uses corrected legacy species flux
DLBFoam/load-balanced chemistry	OF14 standard chemistry with cpuLoad true plus native LoadBalancer	OF14 native substitution	Old DLBFoam dependency intentionally removed
Legacy/custom parallel redistribution	fvMeshDistributors::loadBalancer and Scotch	OF14 native substitution	MPI/restart qualified
Legacy/custom 3-D AMR dependency	OF14 native fvMeshTopoChangers::refiner	OF14 native substitution	Serial/MPI/restart qualified
True 2-D/axisymmetric AMR	New reusable planarRefiner library	New OF14 portable component	Independent of detonationFoam
Published legacy Soret H/H2 behavior	Disabled	Left out / deferred	OF8 H/H2 assignment ambiguity must be resolved explicitly
Automatic unrefinement in planarRefiner	Not implemented	Left out / deferred	Current 2-D component is refinement-only
Full-resolution long OF8/OF14 equivalence and formal grid convergence	Not release-blocking	Deferred post-release qualification	Laptop policy limits release gates to short tests
Formal strong/weak scalability study	Not release-blocking	Deferred post-release qualification	G4 is a compact infrastructure/timing smoke only

9.3 Important migration detail for NS_mixtureAverage

Do not write:

```
solverType NS_mixtureAverage;
```

in the OF14 case. The release intentionally avoids reviving that legacy equation branch. Use the viscous solver path and select the compatibility physics through the thermo/transport dictionaries, for example:

```
// constant/physicalProperties
thermoType
{
    type            hePsiThermo;
    mixture          coefficientWilkeMulticomponentMixture;
    transport        logPolynomial;
    thermo           janaF;
    energy           sensibleInternalEnergy;
    equationOfState  perfectGas;
    specie           specie;
}
```

and:

```
// constant/thermophysicalTransport
laminar
{
    model legacyMixtureAverageFourier;
    legacyThermalDiffusionMode off;
    D
    {
        // one legacyBinaryDiffusionCoefficient entry per binary pair
    }
}
```

10. Qualification summary

The staged migration closed the following areas before R1 packaging:

Stage	Main qualification
A	modular foamRun solver, thermo/chemistry startup, fast OF8/OF14 baseline
B1-B5	HLL, HLLC, HLLCP, AUSM+, AUSM+up restoration
C	seven-flux serial/MPI, native load balancing, restart/redecomposition, fixed-time OF8/OF14 profiles
D	legacy NS_mixtureAverage transport/property audit, exact compatibility model, serial/MPI property regression
E	native OF14 3-D AMR, localized AMR comparison, MPI/load balancing/restart
F	reusable true-2-D planarRefiner, wedge portability, MPI/distributed restart
G1	integrated laptop release regression - 205 s
G2	compact OF8/OF14 equivalence replay - 31 s
G3	mesh/time-step characterization - 249 s
G4	serial/MPI2/MPI4 load-balancer/Scotch smoke - 348 s

Selected G3 sensitivity results at $t = 2.7 \times 10^{-8}$ s:

Mesh/time-step case	Shock [m]	p_{max} [Pa]	T_{max} [K]
10 um, Co 0.10	0.0102634382	1,986,615.73	4994.4747
5 um, Co 0.10	0.0102627687	2,171,901.06	4994.47564
2.5 um, Co 0.10	0.0102627994	2,262,087.22	4994.47605
5 um, Co 0.05	0.0102627957	2,152,715.98	4994.47583

The shock position is already very stable from 5 to 2.5 um, while peak pressure remains more resolution-sensitive. G3 is therefore a screening characterization, not a claim of asymptotic grid convergence.

G4 demonstrated the runtime parallel infrastructure on the 2400-cell HLLCP case:

Run	Wall time [s]	Speedup	Efficiency	Redistrib.
serial	152.966	1.000	1.000	0
MPI2	91.855	1.665	0.833	29
MPI4	102.875	1.487	0.372	29

The small-case speedups are diagnostic only; no release claim of formal parallel scalability is made.

11. Known limitations and post-release work

The following items are intentionally not release-blocking:

- full-resolution, long-duration OF8-to-OF14 equivalence runs;
- formal grid-convergence/CJ-speed validation;
- long endurance runs with repeated AMR/load-balance/restart cycles;
- formal strong/weak parallel scalability studies;
- corrected or forensic-compatible legacy Soret behavior;
- automatic unrefinement/coarsening in planarRefiner.

The release qualification policy targets approximately 30 minutes or less per gate on a laptop. Longer studies are documented in qualification/POST_RELEASE_DEFERRED_QUALIFICATION.md.

12. Troubleshooting

12.1 `foamRun` cannot select `detonationFluid`

Confirm the full build completed and that:

```
ls "$FOAM_USER_LIBBIN/libdetonationFluidSolver.so"
```

exists. Also confirm `system/controlDict` contains `solver detonationFluid;`.

12.2 `legacyMixtureAverageFourier` is not found

Ensure `libdetonationLegacyThermophysicalTransportModels.so` was built and loaded in `controlDict`:

```
libs
(
    "libdetonationLegacyThermophysicalTransportModels.so"
);
```

12.3 AMR library is not found

Build it independently:

```
./AllwmakeAMR
```

and confirm:

```
ls "$FOAM_USER_LIBBIN/libplanarFvMeshTopoChangers.so"
```

12.4 Load balancer says `selecting distributor simple`

The native load balancer reads the active `system/decomposeParDict`. If runtime Scotch redistribution is intended, switch the active dictionary to:

```
method scotch;
```

before `foamRun` starts.

12.5 Localized 2-D AMR creates a few non-hex transition cells

This is expected for the qualified directional local-refinement topology. F3 accepted four bounded 7-face transition polyhedra in the reference slab case while maintaining low non-orthogonality/skewness and species closure. Uniform planar refinement and the wedge portability case remained strict Mesh OK. cases.

13. File map

```
applications/modules/detonationFluid/
    OpenFOAM 14 foamRun detonation solver
src/detonationLegacyThermophysicalTransportModels/
    exact legacy mixture-average/property compatibility library
src/planarFvMeshTopoChangers/
    reusable 2-D slab/wedge AMR library
tutorials/
    migrated NH3/O2 detonation examples and OF8 comparison reference
tools/
    legacy binary-diffusion conversion support
docs/
    this manual and standalone planarRefiner manual in Markdown/Word/PDF
qualification/
    compact gate evidence and deferred post-release qualification notes
```